

JAMES WATT SCHOOL OF ENGINEERING

ENG2083: Introductory Programming 2

SPQR: Design and Implementation of an Extended Duodecimal Roman Numeral Desktop Calculator

Final Programming Lab Report
(Laboratory 8 – Option 6)

Author: Daniel Mui
Student Number: 2962389m
Course Lecturer: Prof. David Flynn
Submitted Code File: `Lab8-2962389m.py` (473 Lines, Python 3.8+ / `tkinter`)
Submission Date: 2026-11-03

This report critically assesses the mathematical and algorithmic challenges of non-positional Roman numeral and duodecimal fractional computation [1, 4], documents the object-oriented software architecture implemented in `Lab8-2962389m.py` (`Lines 001-473`), and evaluates numerical accuracy and real-world human-computer interaction.

1. Mathematical Context & Critical Assessment of the Problem

Designing a calculator for Roman Numerals presents some key challenges rooted in the fundamental difference of our modern math and a non positional system.
Unlike the Arabic place-value system, Roman notation is a bi-quinary (alternating base-5 and base-2) additive-subtractive system comprising seven symbols ($\mathrm{I} = 1, \mathrm{V} = 5, \mathrm{X} = 10, \mathrm{L} = 50, \mathrm{C} = 100, \mathrm{D} = 500, \mathrm{M} = 1000$) [4]. While standard programming implementations restrict conversion to integers between 1 and $3,999$ [2, 3], a complete calculator must address 5 primary deficiencies:

1. **Absence of Place-Value & Subtractive Rules:** To avoid massive strings such as LXXXXVIIII (99), standard notation places a smaller symbol to the left of a larger symbol to indicate subtraction ($\mathrm{IV} = 4, \mathrm{IX} = 9, \mathrm{XL} = 40, \mathrm{XC} = 90, \mathrm{CD} = 400, \mathrm{CM} = 900$) [2, 4]. Lewis [4] gives 3 rules: 1. only I, X, and C may be subtracted; 2. the subtracted numeral must be no less than one-tenth of the symbol it precedes, meaning each power of ten is processed separately (49 is XLIX, never IL; 1999 is MCMXCIX, never MIM); and 3. only a single smaller numeral may precede a larger one.
2. **Ambiguity:** Initially subtractive rules were optional (e.g., Colosseum arches bear XLIIII for 44 and XXVIIII for 29) [4]. Conversely, modern spreadsheet functions such as Microsoft Excel's `=ROMAN(n,4)` invent non-historical contractions or silently accept malformed strings (IIII, VV, IC, XXXX). We shall aim to reject these in favour of a "true" canon output and input [3, 4].
3. **Zero:** Roman notation lacks a symbol for zero because empty powers of ten needed no placeholder outside the empty grooves of an abacus [1, 4]. Subtraction ($\mathrm{L} - \mathrm{L} = 0$ or $\mathrm{X} - \mathrm{XXV} = -15$) then requires the use of the symbol **N (*nulla*)** alongside negative prefixing (-).
4. **Vinculum Notation:** Almost all pre-existing converters terminate at MMMCMXCIX ($3,999$) because fourfold repetition of M is not allowed [2, 3]. Historically, Romans multiplied values by $1,000$ using either a horizontal overline (*Vinculum*, e.g., $\overline{\mathrm{V}} = 5,000$) or 'deep parentheses' (e.g., `(I)` $= 1,000$, `(XXIII)` $= 23,000$) [4]. Adopting this bracket based so called "Vinculum notation" `(V)` gives us both historical conventions in standard ASCII, extending capacity to `(MMMCMXCIX)CMXCIX` ($3,999,999$).
5. **Fractions:** As told by Maher and Makowski [1], Romans avoided base-10 decimals and instead divided into twelfths. Because 12 is divisible by 2, 3, 4, and 6, it allowed exact representation of 1/2, 1/3, 1/4, and 1/6 [1]. As shown by Maher and Makowski [1] and Lewis [4], one-half (semis, $\frac{6}{12}$) was denoted by S, while "uncia" dots or dashes (`.` $= \frac{1}{12}$, `:` $= \frac{2}{12}$) were appended to form all eleven named fractions from uncia ($\frac{1}{12}$) to deunx ($\frac{11}{12}$), but weren't not used subtractively [1, 4].

2. Implementation

The Calculator split using Object-Oriented Programming (OOP) into two classes (**Appendix A, Figure 1**): `CalculatorMain` (`Lines 014-217`), housing the conversion, validation, and arithmetic engine; and `RomanCalculatorGUI` (`Lines 226-460`), managing the `tkinter` desktop interface.

2.1 Greedy and Lookahead Parsing (`Lines 016-078`)

In `CalculatorMain.__init__` (`Lines 016-050`), we use 3 dictionaries to encode the numerals, including the complete 12-entry unciae table `self.twelfths_to_roman_dict` (`Lines 037-050`) mapping 0–11 twelfths to their classical Latin names (*uncia, sextans, quadrans, triens, quincunx, semis, septunx, bes, dodrans, dextans, deunx*) as given in Maher and Makowski [1]. For the standard 1–3,999 domain, `_basic_int_to_roman` (`Lines 071-078`) adapts the greedy dictionary decomposition pattern [3, 4] over the 13 ordered pairs in `self.int_to_roman_dict` (`Lines 026-034`), using division (`count = num // val`, `Line 075`) and modulus reduction. Then within, `_basic_roman_to_int` (`Lines 055-069`) we can refine the traditional 1–3,999 single-pass lookahead algorithm [2, 3], subtracting the current symbol's weight if it precedes a larger (`Lines 065-066`) and reporting a `ValueError` for unrecognised characters.

Vinculum Scaling and Duodecimal (`Lines 080-117`) We overcome the integer limits of standard converters [2, 3], using `number_to_roman` (`Lines 080-117`)which generalises to arbitrary floats and integers:

- **Zero, Sign, and *Unciae* Rounding:** Values near zero return `"N"`. After extracting any negative sign (`Lines 085-086`) and separating the integer from the fractional part (`Lines 088-089`), the fraction is quantized to the nearest twelfth via `twelfths = int(round(frac * 12))` (`Line 092`), carrying into `whole` if `twelfths == 12` (`Lines 093-095`).
- **Vinculum :** Values exceeding $3,999,999$ trigger an `OverflowError` (`Lines 097-098`). Integers $\geq 4,000$ are decomposed into `thousands = whole // 1000` and `remainder = whole % 1000` (`Lines 103-104`), enclosing the thousands multiplier in deep parentheses `"({})"` (`Line 105`) [4] before appending remainder (`Line 107`) and the unciae suffix [1].

2.2 Extended Parsing and Validation (`Lines 121-182`)

The `roman_to_number` method (`Lines 121-182`) parses any extended strings in four stages: (1) whitespace stripping and `"N"` handling (`Lines 126-131`); (2) negative sign extraction (`Lines 134-138`); (3) right-to-left extraction of trailing duodecimal symbols (`'S'`, `':'`, `'.'`) via a `while` loop (`Lines 140-155`), summing twelfths additively as described by Maher and Makowski [1] and rejecting improper fractions; and (4) parenthetical *Vinculum* slicing (`Lines 160-173`), scaling the bracketed prefix by $1,000$ (`Line 169`) [4].

Because standard lookahead [2] alone woudl accept strings that violate Lewis's [4] three subtractive rules (e.g., `"IIII"`, `"VV"`, or `"IC"`), we use `Lines 178-180` to enforce **Canonical Validation**:

```
178         canonical = self.number_to_roman(total)
179         if canonical != raw_input:
180             raise ValueError("not valid roman notation")
```

Sending total through `number_to_roman` yields the 'proper' or legal numeral for any value in $[-3999999\frac{11}{12},\ +3999999\frac{11}{12}]$. Any other non-canon, including 4x repetition (`IIII`), illegal subtractive jumps (`IC`, `MIM`) [4], redundant brackets (`(I)` for `M`), or unreduced *unciae* (`:::` for `S`) [1]—fails `Line 179` and raises `"not valid roman notation"` (`Line 180`).

3. Verification

The `evaluate` block (`Lines 188-217`) executes binary and unary operations while preventing division by 0 (`Lines 209-210`) and negative square roots (`Lines 193-194`).

4. Real-World Use and UX

Implemented in `RomanCalculatorGUI` (`Lines 226-460`), the desktop interface uses a Texas Instruments (TI-89) styled 2D button grid (`Lines 277-307`; **Appendix B**). On every click, `_update_displays` (`Lines 338-357`) updates the white Roman LCD (`Line 259`) alongside a live Arabic bar (`Lines 353-357`) that renders valid decimal translations in green or displays `"not valid Roman notation"` in red. In the real world, this tool has limited helpfulnes: but may find use in Computer Science, to illustrate how non-positional parsing and duodecimal rounding operate; as well in Archival uses to prevent historically inaccurate non-canonical errors during the transposing of texts.

Bibliography

[1] David W. Maher and John F. Makowski. 2001. Literary Evidence for Roman Arithmetic with Fractions. *Classical Philology* 96, 4 (October 2001), 376–399. https://doi.org/10.1086/449557

[2] GeeksforGeeks. 2021. Python Program for Converting Roman Numerals to Decimal Lying between 1 To 3999. *GeeksforGeeks*. Retrieved November 3, 2026 from https://www.geeksforgeeks.org/python/python-program-for-converting-roman-numerals-to-decimal-lying-between-1-to-3999/

[3] Bas Steins. 2022. Coding Interview – Converting Roman Numerals in Python. *Bas codes*. Retrieved November 3, 2026 from https://bas.codes/posts/python-roman-numerals

[4] Paul Lewis. 2019. Roman Numerals - History and Use. *Clarahost.co.uk*. Retrieved November 3, 2026 from http://www.web40571.clarahost.co.uk/roman/howtheywork.htm

A. Software Architecture & Canonical Validation Flowchart

Figure 1 illustrates the architecture connecting `RomanCalculatorGUI` (`Lines 226-460`) and `CalculatorMain` (`Lines 014-217`), detailing the four-stage pipeline and the **Validation** check (`Lines 178-180`) enforcing the rules given by Maher and Makowski [1] and Lewis [4].

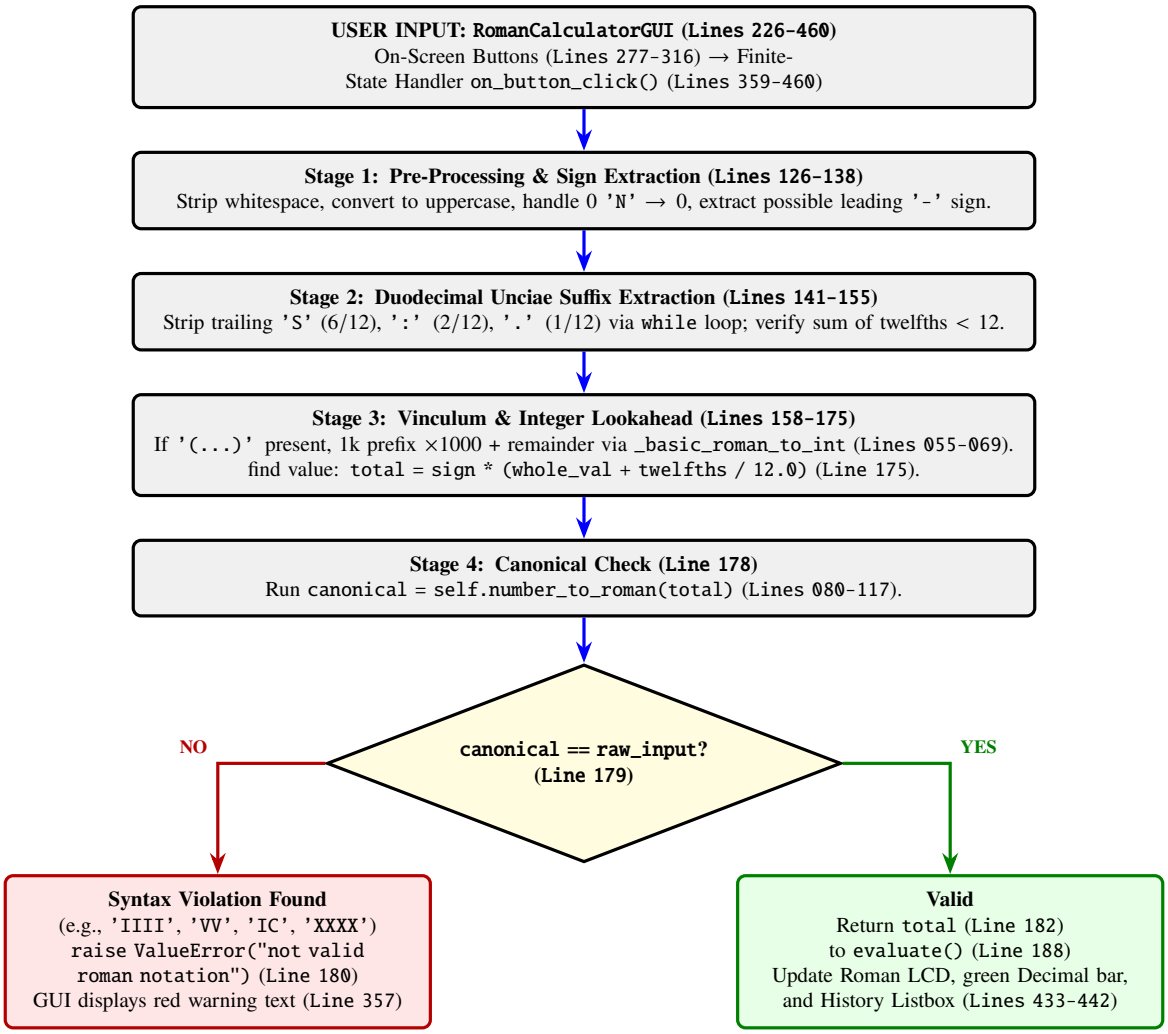


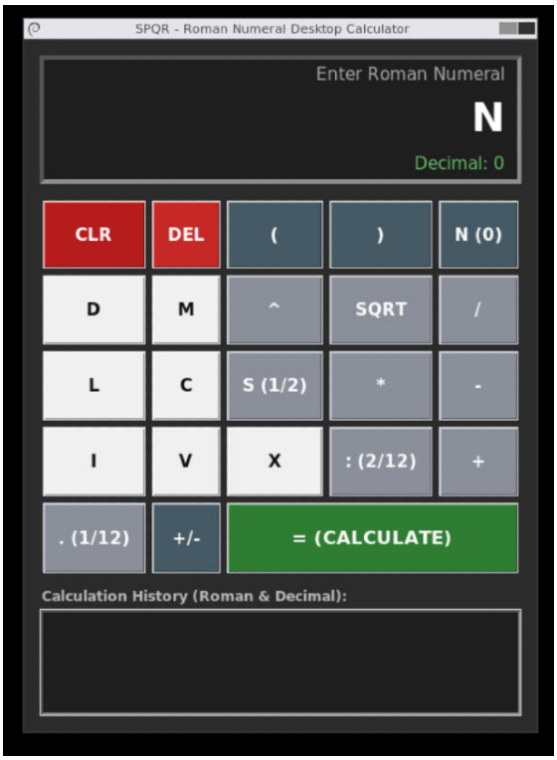
Figure 1: Execution flow of the Calculator, highlighting the decoupling of the `tkinter` GUI state machine from the validation engine.

B. Annotated Screenshots

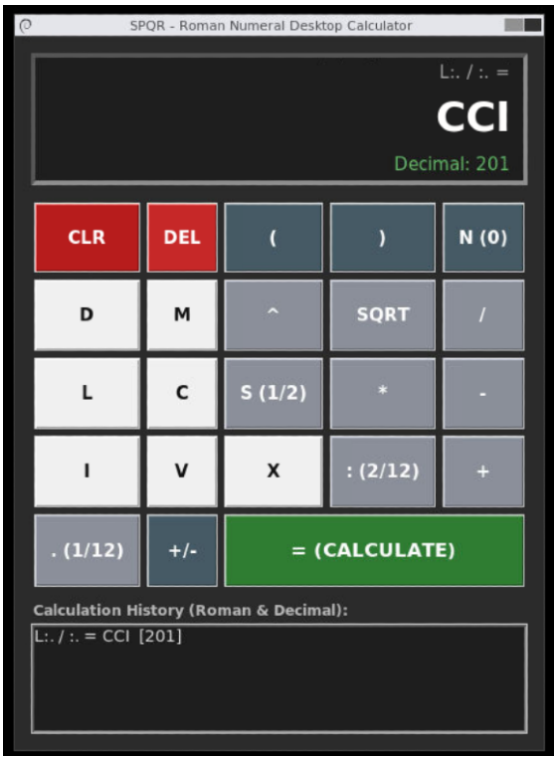
Figures 2a and 2b illustrate the operational `tkinter` desktop interface (`RomanCalculatorGUI`, `Lines 226-460`) in its initial ready state and during compound duodecimal fraction division.

C. Complete Engine Verification & Boundary-Test Matrix

Table 1 documents the systematic verification of `CalculatorMain.evaluate()` (`Lines 188-217`) and `CalculatorMain.roman_to_nu` (`Lines 121-182`) across classical literary test cases [1], standard 1–3,999 conversion benchmarks [2, 3], extended numeral domains [4], and invalid syntax traps.



(a) **Initial State** Upon launch (`Lines 247-269`), the calculator initialises to Nulla which acts as 0. The keypad (`Lines 277-316`) groups controls by function using TI-84/TI-89 high-contrast ergonomics: red clear/delete keys (`#b71c1c`), charcoal Vinculum/sign/zero keys (`#455a64`), white numeral keys (`#f0f0f0`), grey operators/fractions (`#8a8f99`), and a 3-column green calculate bar (`#2e7d32`).



(b) **Division:** Demonstrates exact compound duodecimal division (`Lines 422-445`) using the *quadrans* ($\frac{3}{12}$, `:.`) notation documented in [1]: $50\frac{3}{12}$ (`L:. = 50.25`) divided by $\frac{3}{12}$ (`:. = 0.25`) yields `CCI` (201). The current expression (`L:. / :. =`), the main Roman result (`CCI`), the live decimal translation (`Decimal: 201`), and the Calculation History log update simultaneously.

Figure 2: Operational screenshots of the Calculator GUI.

Table 1: Systematic Verification Matrix for `CalculatorMain` (`Lines 014-217`)

Test ID	Input Expression / String	Engine Output (`Roman` / `Decimal`)	Target Mechanism & Reference Verified
T01	`XIV + XXVI`	`XL (40)`	Standard additive/subtractive synthesis (`Lines 071-078`) [3, 4]
T02	`MCMXCIV + VI`	`MM (2000)`	Subtractive lookahead & carry propagation [2, 3]
T03	`L - L`	`N (0)`	Zero identity (*Nulla*) handling (`Lines 082-083`)
T04	`X - XXV`	`-XV (-15)`	Signed negative Roman domain (`Lines 085-086`)
T05	`C * L`	`(V) (5000)`	Automatic *Vinculum* scaling above 3,999 (`Lines 102-105`) [4]
T06	`(V) + D`	`(V)D (5500)`	Deep parentheses / *Vinculum* input parsing (`Lines 160-171`) [4]
T07	`::. - . / ::. + .`	`:: (0.333) / S (0.5)`	Horace *Ars Poetica unciae* drills ($\frac{5}{12} \mp \frac{1}{12}$) [1]
T08	`V * IS:.`	`VIIIS:. (8.75)`	Frontinus *De aquaeductu* area ($5 \times 1\frac{3}{4} = 8\frac{3}{4}$) [1]
T09	`L:. / :.`	`CCI (201)`	Exact compound *unciae* division ($50.25 \div 0.25$) [1]
T10	`X / III`	`III:: (3.333)`	Base-12 *triens* quantization ($3\frac{4}{12}$, `Lines 092-095`) [1]
T11	`XII ^ III`	`MDCCXXVIII (1728)`	Exponentiation (`^`, `Lines 212-213`)
T12	`IIII`, `VV`, `IC`, `XXXX`	`ValueError: not valid roman notation`	Canonical Round-Trip Validation (`Lines 178-180`) [4]